VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

# PROJECT REPORT

**CS163**

*Group members:*

1. 25125052 - Võ Thành Đạt
2. 25125055 - Phạm Huy Khánh
3. 25125068 - Lê Phi Truyền
4. 25125089 - Lê Trọng Đại Nghĩa

28th April 2026

# Contents

**1 Abstract** **2**

**2 Introduction** **3**

**3 Project Architecture** **4**

3.1 Folder Structure . . . . . 4

3.2 Design Patterns . . . . . 5

3.3 High-Level Diagram . . . . . 6

**4 Implementation Details** **7**

4.1 Core Structures and Methods . . . . . 7

4.2 Main Flows and Use Cases . . . . . 8

**5 Technical Problems and Solutions** **10**

5.1 Fragmentation of Project Architecture . . . . . 10

5.2 Tight Coupling and Component Interdependency . . . . . 10

**6 Conclusion** **11**

6.1 Project Achievements . . . . . 11

6.2 Project Strengths . . . . . 11

6.3 Future Enhancements . . . . . 12

**7 References** **13**

**8 Appendices** **13**

# 1 Abstract

This report presents the comprehensive design and implementation of the **DSA Visualizer**, an interactive tool developed in **C++** using the **SFML** library. The project aims to enhance the understanding of complex algorithms through real-time visualization of four distinct areas: **Singly Linked Lists**, **Hash Table (Chaining)**, **Tries (Prefix Trees)**, and **Minimum Spanning Trees (MST)**. Adhering to a strict architectural separation of concerns, the system isolates pure logic from graphical rendering, ensuring high maintainability and performance. By utilizing a frame-independent animation system and a centralized **ScreenManager**, the application provides a smooth, interactive experience for exploring data operations and graph algorithms. This summary highlights the design choices, the **technical** challenges of synchronizing states, and the final results achieved by the group.

# 2 Introduction

In the field of computer science education, understanding complex data structures and algorithms (DSA) often poses a significant challenge for learners. Traditional teaching methods, which rely heavily on static diagrams and theoretical explanations, can make it difficult for students to fully grasp dynamic processes such as pointer manipulation, hashing, tree traversal, and graph optimization. As a result, there is a growing need for more interactive and intuitive learning tools that can bridge the gap between theory and practice.

Motivated by this challenge, this project presents the design and development of a **DSA Visualizer**, an interactive application built using **C++** and the **SFML (Simple and Fast Multimedia Library)** framework. The primary goal of this project is to enhance the learning experience by providing real-time visual representations of fundamental data structures and algorithms. By allowing users to observe how operations are executed step by step, the system transforms abstract concepts into tangible and engaging visual experiences.

The visualizer focuses on four key areas: **Singly Linked Lists**, **Hash Tables**, **Tries (Prefix Trees)**, and **Minimum Spanning Trees (MST)**. These structures were carefully selected due to their importance in both academic curricula and real-world applications. Through interactive simulations, users can perform operations such as insertion, deletion, searching, and traversal, while simultaneously observing how the underlying data structure evolves.

This project is important because it not only improves conceptual understanding but also promotes active learning. By integrating a clear architectural separation between core logic and graphical rendering, the system ensures maintainability, scalability, and performance efficiency. Ultimately, the DSA Visualizer aims to serve as a practical educational tool that empowers students to explore, experiment, and develop a deeper intuition for algorithmic thinking.

# 3 Project Architecture

## 3.1 Folder Structure

This project utilizes a Modular, Layered Architecture heavily inspired by the Model-View-Controller pattern. The primary goal of this architecture is to keep the mathematical logic of data structures completely independent from the graphics rendering and user interface.

```
/src
  main.cpp                        # Entry point, simply starts the Core::App
  /Command                        # Command design pattern
    ICommand.h                    # Base abstract class to inherit
    OperationCommand.h / cpp      # Command for operation menu in visualization scene
    PlaybackCommand.h / cpp       # Command for playback control widget
    SceneCommand.h / cpp          # Command to switch scene
  /Core
    App.h / cpp                   # Contains the main sf::RenderWindow and game loop
    SceneManager.h / cpp          # Handles transitioning between the 4 visualizer scenes
    ResourceManager.h / cpp       # Singleton pattern
  /DataStructures                 # PURE C++ LOGIC
    LinkedList.h / cpp
    HashTable.h / cpp
    Trie.h / cpp
    Kruskal.h / cpp
  /Scene
    BaseScene.h / cpp             # Base abstract class to inherit
    VisualizationScene.h / cpp    # Base abstract class for visualizer scene
    MenuScene.h / cpp             # Main menu scene
    LinkedListScene.h / cpp       # Linked list scene
    HashTableScene.h / cpp        # Hash table scene
```

```
    TrieScene.h / cpp              # Trie scene
    KruskalScene.h / cpp           # Kruskal scene
  /UI
    Button.h / cpp                 # Reusable clickable UI elements
    ButtonMenu.h / cpp             # List of buttons
    OperationMenu.h / cpp          # Operation menu for visualizer
    PlaybackControlWidget.h / cpp  # Control the flow of visualizer
    PlaybackSpeed.h / cpp          # Adjust the speed of visualizer
    SelectBox.h / cpp              # Select box
    TextBox.h / cpp                # Text box
    TextBoxGroup.h / cpp           # List of text boxes
  /Visualizers                     # Visualizer drawing
    BaseVisualizer.h               # The abstract interface everyone must use
    LinkedListVisualizer.h / cpp   # Linked list visualizer
    HashTableVisualizer.h / cpp    # Hash table visualizer
    TrieVisualizer.h / cpp         # Trie visualizer
    KruskalVisualizer.h / cpp      # Kruskal visualizer
```

## 3.2 Design Patterns

- The model layer (Logic & Data):
  - `DataStructures/`: This folder contains pure C++ logic without any SFML dependencies. By isolating the core algorithms from the graphics library, the data structures remain robust, highly testatble and resuable.
- The controller layer (Engine & Flow):
  - `Core/`: Acts as the engine of the visualizer. `App.cpp` manages the main application loop (handling event, update and render), while `SceneManager.cpp` manage transitions between different visualizer screens.

- **Command/**: This folder implements the Command design pattern to bridge user inputs with system actions. Specifically, `SceneCommand.cpp` creates command objects that allow UI Buttons to interact directly with the `SceneManager` to trigger scene transitions. Similarly, `OperationCommand.cpp` generates commands that connect button clicks to the specific data structure operations within the visualizers. This pattern completely decouples the UI components from the complex execution logic.

- The view layer (Rendering & Interaction):
  - `Scene/`: Acts as the main structural container that binds a specific mathematical data structure to its corresponding user interface and visualizer.
  - `Visualizers/`: These classes act as translators. They read the state from the `DataStructures` and convert them into shapes and animations for the scene.
  - `UI/`: Reusable library or interactive components (Buttons, TextBoxes).

## 3.3 High-Level Diagram

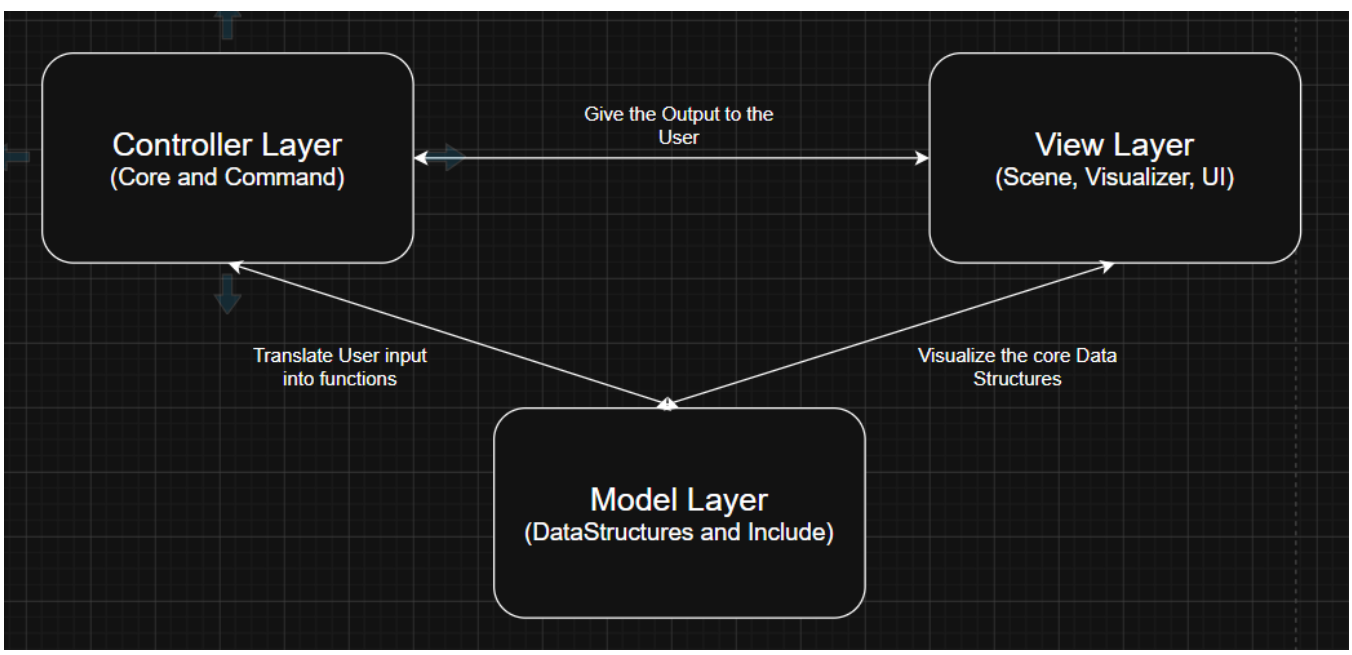


Figure 1: High-level architecture diagram of the system.

# 4 Implementation Details

This section outlines the core components of the visualizer's architecture and describes how these structures and methods collaborate dynamically to execute the primary features of the application.

## 4.1 Core Structures and Methods

To maintain a clean separation of concerns, the project relies on distinct classes categorized by their architectural roles. The following list highlights the most critical classes and their primary functions:

- **Data Logic (Pure Data Structures)**
  - **Classes:** `LinkedList`, `HashTable`, `Trie`, `Kruskal`.
  - **Key Methods:** `insert(value)`, `remove(value)`, `search(value)`, `update(key, value)`. These methods execute the mathematical operations and update the internal state necessary for the visualizer to generate accurate animations.
- **The Visualizers (Graphics & Animation)**
  - **Classes:** `BaseVisualizer`, `HashTableVisualizer`, `LinkedListVisualizer`, `TrieVisualizer`, `KruskalVisualizer`.
  - **Key Methods:** `update(deltaTime)` (calculates animation frame interpolation) and `render(sf::RenderWindow&)` (renders the calculated shapes and text to the active SFML window).
- **The Command System (Interactivity)**
  - **Classes:** `ICommand`, `OperationCommand`, `SceneCommand`.
  - **Key Methods:** `execute()`. This abstract method is strictly overridden by derived classes to trigger data structure operations or request scene transitions without coupling the UI directly to the execution logic.

- **Core Engine (Application Flow)**
  - **Classes:** `App`, `ScreenManager`.
  - **Key Methods:** `run()` (maintains the primary game loop for event polling, updating, and rendering) and `pushScene(BaseScene)`/`popScene()` (handles the transition of different visualization screens).
- **Singleton Pattern**
  - **Classes:** `ResourceManager`.
  - **Key Methods:** `getTexture(alias)` (returns a loaded 2D texture based on its string alias) and `getFont(alias)` (returns a loaded `sf::Font` object based on its string alias).

## 4.2 Main Flows and Use Cases

The application's core workflow is driven by the `App::run()` loop, which continuously executes three main phases every frame: `processEvents()`, `update(deltaTime)`, and `render()`. These calls are delegated to the `SceneManager`, which routes them exclusively to the currently active scene (the scene at the top of its internal stack). The following use cases demonstrate how the application operates and how functions interact during real-time user executions.

**Use Case 1: Scene Transition (Navigating from Main Menu to Linked List)**
This use case illustrates how the application navigates between different interfaces using the stack-based `SceneManager`.

1. **Event Capture:** While the `MenuScene` is active, the `App` polls a mouse click event. The `App` delegates this event via `SceneManager::processEvents()` down to the `MenuScene`.
2. **Button Trigger:** The event is passed to the `ButtonMenu` UI component. Detecting a click on the "Linked List" button, it executes its bound callback command.

3. **Executing Command:** The bound command, created by `createLinkedListSceneCommand()`, invokes `SceneManager::pushScene(std::make_unique<LinkedListScene>())`.

4. **State Update:** The `SceneManager` pushes the newly instantiated `LinkedListScene` onto its internal stack.

5. **Render Active Scene:** On the next iteration of the `App::run()` loop, the `SceneManager` delegates `update()` and `render()` calls to the new top of the stack (`LinkedListScene`), effectively completing the screen transition.

**Use Case 2: Inserting a Value into a Linked List**

This use case demonstrates the data flow when a user interacts with a data structure, highlighting the separation of concerns between the UI, the Scene, and the Visualizer logic.

1. **User Input:** Inside the `LinkedListScene`, the user types a value into the `OperationMenu` input box, selects "At Tail", and clicks the "Insert" button.

2. **Event Delegation:** The `OperationMenu` handles the click event and invokes the scene's designated callback function: `LinkedListScene::onInsert(value)`.

3. **Updating UI State:** Inside `onInsert()`, the scene parses the input and determines the operation type. It immediately calls `PseudocodePanel::setPseudocode()` to display the relevant algorithm's code on the screen.

4. **Visualizer Logic:** The scene then delegates the core execution to the visualizer by calling `LinkedListVisualizer::insertValue()`.

5. **Animation Step Generation:** The visualizer performs the algorithmic insertion on its internal `LinkedList` data structure. Instead of instantly rendering the change, it pre-calculates and generates a sequence of `InsertStep` objects. Each step dictates the node states, spatial transformations, and the specific `pseudocodeLine` index to be highlighted.

6. **Continuous Rendering:** Over subsequent frames, `LinkedListVisualizer::update(deltaTime)` advances through the pre-calculated steps based on the user's playback speed. Simultaneously, `LinkedListVisualizer::render()` continuously draws the `VisualNode` elements as they animate towards their new positions and colors.

# 5 Technical Problems and Solutions

While building our visualization project, we faced multiple technical obstacles. We overcame these through collaborative research and creative problem-solving. Below, we detail the specific challenges encountered and the solutions we developed to move the project forward.

## 5.1 Fragmentation of Project Architecture

**The Problem**: Because team members developed different data structures independently, the initial codebase lacked a unified architecture. This resulted in inconsistent naming conventions, disparate file directories, and conflicting logic, making it extremely difficult to integrate individual components into the main visualization engine.

**The Solution**: We temporarily paused feature development to establish a strict, standardized project skeleton. We implemented a core interface that all data structures were required to inherit from, reorganized the repository into a clear Model-View-Controller (MVC) directory structure.

## 5.2 Tight Coupling and Component Interdependency

**The Problem**: As the application grew, we discovered that our files were deeply intertwined. The data structure logic was tightly coupled with the visualization rendering, meaning that a seemingly isolated change in one file—such as modifying a node's internal state or updating a search algorithm—would often cause cascading errors and break unrelated components across the engine.

**The Solution**: We refactored the codebase to enforce strict encapsulation and modularity. By defining clear API boundaries and decoupling the data models from the UI components, we

minimized hidden dependencies. We also introduced an event-driven update system, ensuring that internal file modifications remained isolated and did not produce unintended side effects elsewhere in the project.

# 6 Conclusion

## 6.1 Project Achievements

The development of the Data Structures and Algorithms (DSA) Visualizer has been successfully completed, meeting the core objectives outlined in the initial requirements. The application provides a highly interactive, responsive, and educational platform for visualizing four distinct types of data structures: **Linked List**, **Hash Table (Chaining)**, **Trie**, and **Graph (Kruskal's Minimum Spanning Tree)**.

By building a custom UI framework on top of SFML from scratch, the project successfully implements complex features such as real-time animation, synchronized pseudocode highlighting, and a step-by-step playback system.

## 6.2 Project Strengths

The success of this project lies heavily in its robust technical foundation and user-centric design:

- **Robust Software Architecture:** The project strictly adheres to Object-Oriented Programming (OOP) principles. By utilizing design patterns such as *Command*, *Singleton*, and *Factory*, the codebase is highly modular. The strict separation between Data Logic (Backend), Visualizers (Rendering), and Scenes (UI Controller) makes the application extremely stable and easy to maintain.

- **Advanced Snapshot System:** Instead of relying on fragile timer-based animations, the implementation of the *Snapshot (Memento) Pattern* allows users to effortlessly navigate back and forth between algorithmic steps (Undo/Redo) without any data corruption.

- **Exceptional User Experience (UX):** The custom UI includes smooth hover effects, dynamic rounded shapes (`sf::ConvexShape`), pan/zoom functionalities, and detailed mathematical breakdowns (e.g., displaying ASCII sum calculations in the Hash Table).
- **Memory Safety:** The extensive use of modern C++17/C++20 features, particularly smart pointers (`std::unique_ptr`, `std::shared_ptr`) and `std::optional`, ensures zero memory leaks throughout the application's lifecycle.

## 6.3 Future Enhancements

While the current version is highly functional, several potential areas for improvement have been identified for future iterations:

1. **Expanding Collision Resolution Methods:** For the Hash Table, adding *Linear Probing* and *Quadratic Probing* alongside the current Chaining method. The underlying `IHashTable` interface has already been structured to easily accommodate this.
2. **Additional Data Structures:** Integrating complex trees such as AVL Trees, Red-Black Trees, and B-Trees, as well as pathfinding algorithms (Dijkstra, A*) for the Graph section.
3. **Data Import/Export:** Allowing users to initialize data structures by uploading external files (CSV or text files) instead of manual input.
4. **Theming Options:** Implementing a global theme manager to allow users to toggle between Light Mode and Dark Mode for better accessibility.

# 7 References


1. **SFML Development Team.** *Simple and Fast Multimedia Library (SFML) Official Documentation*. Retrieved from `https://www.sfml-dev.org/learn.php`. (Utilized for rendering 2D graphics, handling window events, and managing UI shapes).

2. **CS 1332 Data Structures  Algorithms Visualization Tool** *Visualising data structures and algorithms through animation*. Retrieved from `https://csvistool.com/`. (Served as a primary UX/UI inspiration for step-by-step execution and pseudocode synchronization).

3. **Google AI Studio (2024).** *Gemini Large Language Models*. Utilized as an advanced pair-programming assistant for C++ debugging, software design pattern optimization (Snapshot/Memento Pattern), and LaTeX documentation structuring.

4. **Linked List Icon**. Retrieved from `https://thenounproject.com/icon/linked-list-724304/`.

5. **Hash Table Icon**. Retrieved from `https://www.flaticon.com/free-icon/list_15235919?term=linked+list&page=1&position=2&origin=search&related_id=15235919`.

6. **Trie Icon**. Retrieved from `https://www.techcrashcourse.com/2023/08/trie-data-structure.html`.

7. **Kruskal's Algorithm**. Retrieved from `https://algorithms.discrete.ma.tum.de/graph-algorithms/mst-kruskal/img/kruskal_kette.png`.


# 8 Appendices

- Github repo: `https://github.com/khanh47/DSA_Visualizer`
- Demo video: `https://www.youtube.com/watch?v=jGCBeGTbQzU`